exercise 6

siyuzhou

2026-03-24

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com (http://rmarkdown.rstudio.com).

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

```
library(tidyverse) # loads dplyr, ggplot2, and others
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✔ dplyr     1.2.0     ✔ readr     2.2.0
## ✔ forcats   1.0.1     ✔ stringr   1.6.0
## ✔ ggplot2   4.0.2     ✔ tibble    3.3.1
## ✔ lubridate 1.9.5     ✔ tidyr     1.3.2
## ✔ purrr     1.2.1
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to bec
ome errors
```

```
library(stringr) # to handle text elements
library(tidytext) # includes set of functions useful for manipulating text
library(ggthemes) # to make your plots look nice
library(text2vec) # for word embedding implementation
library(widyr) # for reshaping the text data
library(RSpectra) # for svd
library(umap) # for dimensionality reduction
```

```
twts_sample <- readRDS("/Users/aitingrnb/Desktop/socialresearch/computational_text_analysis/ex
ercise6/twts_corpus_sample.rds")

#create tweet id
twts_sample$postID <- row.names(twts_sample)
```

```
tidy_skipgrams <- twts_sample %>%
    unnest_tokens(ngram, tweet, token = "ngrams", n = 6) %>%
    mutate(ngramID = row_number()) %>%
    tidyr::unite(skipgramID, postID, ngramID) %>%
    unnest_tokens(word, ngram)

head(tidy_skipgrams, n=20)
```

```
## # A tibble: 20 × 7
##    date       username     party_value  reftweet_username ments skipgramID word
##    <date>     <chr>        <chr>        <chr>             <chr> <chr>      <chr>
##  1 2017-10-17 leicesterliz Labour       HPIAndyCowper     ""    1_1        colo…
##  2 2017-10-17 leicesterliz Labour       HPIAndyCowper     ""    1_1        me
##  3 2017-10-17 leicesterliz Labour       HPIAndyCowper     ""    1_1        surp…
##  4 2017-10-17 leicesterliz Labour       HPIAndyCowper     ""    1_1        https
##  5 2017-10-17 leicesterliz Labour       HPIAndyCowper     ""    1_1        t.co
##  6 2017-10-17 leicesterliz Labour       HPIAndyCowper     ""    1_1        7xvd…
##  7 2019-05-14 david_duguid Conservative Douglas4Moray     "@Ke… 2_2        great
##  8 2019-05-14 david_duguid Conservative Douglas4Moray     "@Ke… 2_2        stuff
##  9 2019-05-14 david_duguid Conservative Douglas4Moray     "@Ke… 2_2        kenc…
## 10 2019-05-14 david_duguid Conservative Douglas4Moray     "@Ke… 2_2        doug…
## 11 2019-05-14 david_duguid Conservative Douglas4Moray     "@Ke… 2_2        and
## 12 2019-05-14 david_duguid Conservative Douglas4Moray     "@Ke… 2_2        the
## 13 2019-05-14 david_duguid Conservative Douglas4Moray     "@Ke… 2_3        stuff
## 14 2019-05-14 david_duguid Conservative Douglas4Moray     "@Ke… 2_3        kenc…
## 15 2019-05-14 david_duguid Conservative Douglas4Moray     "@Ke… 2_3        doug…
## 16 2019-05-14 david_duguid Conservative Douglas4Moray     "@Ke… 2_3        and
## 17 2019-05-14 david_duguid Conservative Douglas4Moray     "@Ke… 2_3        the
## 18 2019-05-14 david_duguid Conservative Douglas4Moray     "@Ke… 2_3        water
## 19 2019-05-14 david_duguid Conservative Douglas4Moray     "@Ke… 2_4        kenc…
## 20 2019-05-14 david_duguid Conservative Douglas4Moray     "@Ke… 2_4        doug…
```

```
#calculate probabilities
skipgram_probs <- tidy_skipgrams %>%
    pairwise_count(word, skipgramID, diag = TRUE, sort = TRUE) %>% # diag = T means that we al
so count when the word appears twice within the window
    mutate(p = n / sum(n))

head(skipgram_probs[1000:1020,], n=20)
```

```
## # A tibble: 20 × 4
##    item1      item2          n         p
##    <chr>      <chr>      <dbl>     <dbl>
##  1 us         the         4084 0.0000530
##  2 the        us          4084 0.0000530
##  3 by         of          4081 0.0000530
##  4 of         by          4081 0.0000530
##  5 more       the         4080 0.0000529
##  6 the        more        4080 0.0000529
##  7 meet       to          4079 0.0000529
##  8 to         meet        4079 0.0000529
##  9 vote       a           4078 0.0000529
## 10 a          vote        4078 0.0000529
## 11 case       case        4078 0.0000529
## 12 question   question    4076 0.0000529
## 13 no         the         4072 0.0000528
## 14 the        no          4072 0.0000528
## 15 plans      plans       4068 0.0000528
## 16 have       of          4063 0.0000527
## 17 of         have        4063 0.0000527
## 18 we         of          4057 0.0000526
## 19 of         we          4057 0.0000526
## 20 investment investment  4056 0.0000526
```

```
#calculate unigram probabilities (used to normalize skipgram probabilities later)
unigram_probs <- twts_sample %>%
    unnest_tokens(word, tweet) %>%
    count(word, sort = TRUE) %>%
    mutate(p = n / sum(n))
```

```
#normalize skipgram probabilities
normalized_prob <- skipgram_probs %>%
    filter(n > 20) %>% #filter out skipgrams with n <=20
    rename(word1 = item1, word2 = item2) %>%
    left_join(unigram_probs %>%
                  select(word1 = word, p1 = p),
              by = "word1") %>%
    left_join(unigram_probs %>%
                  select(word2 = word, p2 = p),
              by = "word2") %>%
    mutate(p_together = p / p1 / p2)

normalized_prob %>%
    filter(word1 == "brexit") %>%
    arrange(-p_together)
```

```
## # A tibble: 1,040 × 7
##    word1  word2                      n           p      p1         p2 p_together
##    <chr>  <chr>                  <dbl>       <dbl>   <dbl>      <dbl>      <dbl>
##  1 brexit brexit                 38579 0.000501    0.00278 0.00278          64.8
##  2 brexit scotlandsplaceineurope    21 0.000000273 0.00278 0.00000186       52.7
##  3 brexit softer                    44 0.000000571 0.00278 0.00000410       50.1
##  4 brexit blindfold                 50 0.000000649 0.00278 0.00000559       41.8
##  5 brexit dividend                 101 0.00000131  0.00278 0.0000127        37.2
##  6 brexit scenarios                 43 0.000000558 0.00278 0.00000633       31.7
##  7 brexit botched                  102 0.00000132  0.00278 0.0000153        31.2
##  8 brexit deadlock                 143 0.00000186  0.00278 0.0000246        27.2
##  9 brexit negotiator                40 0.000000519 0.00278 0.00000894       20.9
## 10 brexit thwart                    24 0.000000311 0.00278 0.00000559       20.1
## # ℹ 1,030 more rows
```

```
pmi_matrix <- normalized_prob %>%
    mutate(pmi = log10(p_together)) %>%
    cast_sparse(word1, word2, pmi)

#remove missing data
pmi_matrix@x[is.na(pmi_matrix@x)] <- 0
#run SVD
pmi_svd <- svds(pmi_matrix, 256)

glimpse(pmi_matrix)
```

```
## Formal class 'dgCMatrix' [package "Matrix"] with 6 slots
##   ..@ i       : int [1:350958] 0 1 2 3 4 5 6 7 8 9 ...
##   ..@ p       : int [1:21263] 0 7823 14312 20145 25508 29921 34422 39290 43493 46519 ...
##   ..@ Dim     : int [1:2] 21262 21262
##   ..@ Dimnames:List of 2
##   .. ..$ : chr [1:21262] "the" "to" "and" "of" ...
##   .. ..$ : chr [1:21262] "the" "to" "and" "of" ...
##   ..@ x       : num [1:350958] 0.6524 -0.01388 -0.00807 0.27447 -0.52374 ...
##   ..@ factors : list()
```

```
#next we output the word vectors:
word_vectors <- pmi_svd$u
rownames(word_vectors) <- rownames(pmi_matrix)

dim(word_vectors)
```

```
## [1] 21262   256
```

```
nearest_words <- function(word_vectors, word){
  selected_vector = word_vectors[word,]
  mult = as.data.frame(word_vectors %*% selected_vector) #dot product of selected word vector
and all word vectors

  mult %>%
  rownames_to_column() %>%
  rename(word = rowname,
         similarity = V1) %>%
    anti_join(get_stopwords(language = "en")) %>%
  arrange(-similarity)

}

boris_synonyms <- nearest_words(word_vectors, "boris")
```

```
## Joining with `by = join_by(word)`
```

```
brexit_synonyms <- nearest_words(word_vectors, "brexit")
```

```
## Joining with `by = join_by(word)`
```

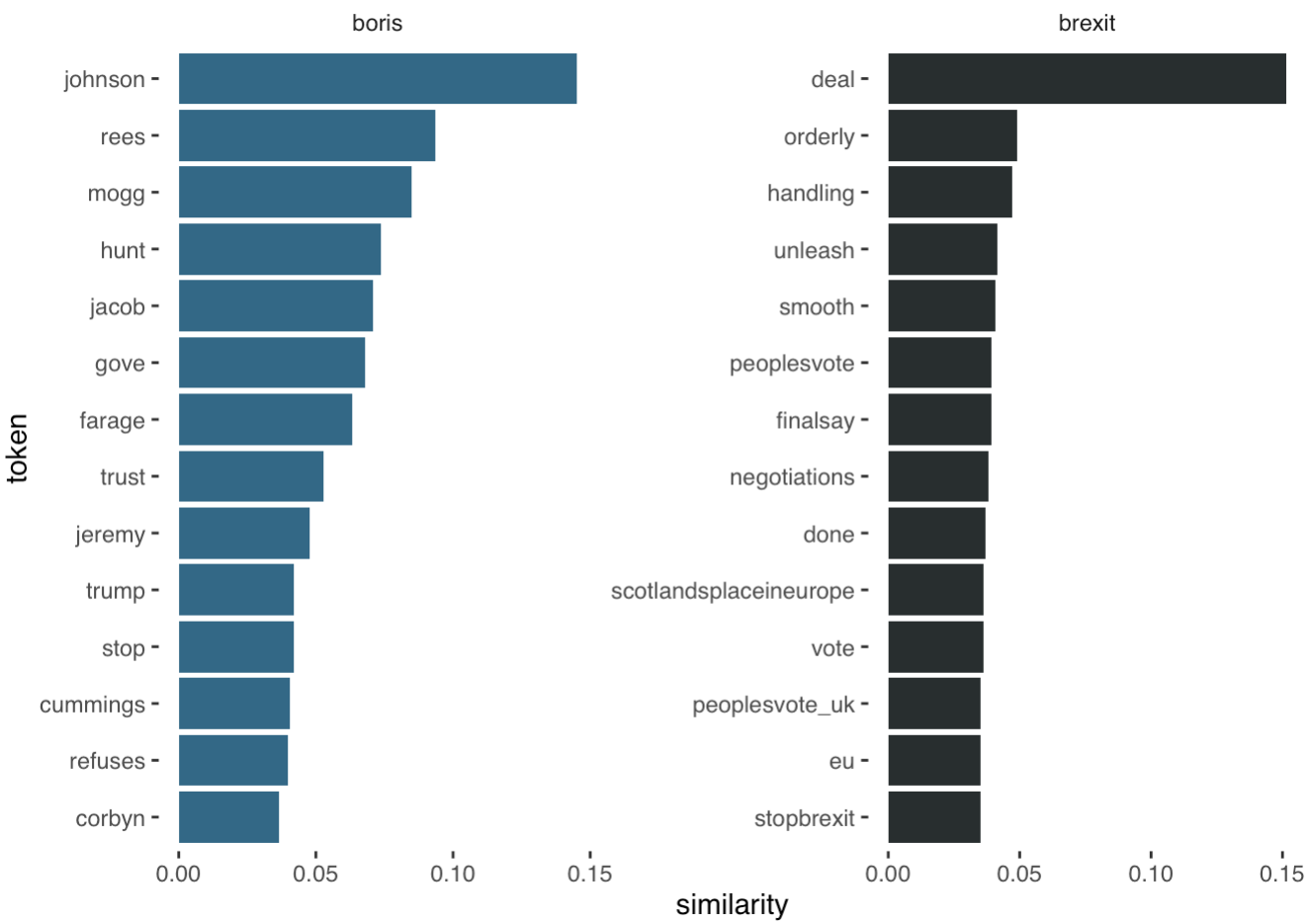
```
head(boris_synonyms, n=10)
```

```
##       word similarity
## 1    boris 0.14916017
## 2  johnson 0.14518169
## 3     rees 0.09371331
## 4     mogg 0.08487125
## 5     hunt 0.07374134
## 6    jacob 0.07096546
## 7     gove 0.06793504
## 8   farage 0.06332522
## 9    trust 0.05281360
## 10  jeremy 0.04771740
```

```
head(brexit_synonyms, n=10)
```

```
##            word similarity
## 1        brexit 0.40700458
## 2          deal 0.15152673
## 3       orderly 0.04909371
## 4      handling 0.04724013
## 5       unleash 0.04151867
## 6        smooth 0.04064841
## 7   peoplesvote 0.03933406
## 8      finalsay 0.03932005
## 9  negotiations 0.03792947
## 10         done 0.03708709
```

```
#then we can visualize
brexit_synonyms %>%
  mutate(selected = "brexit") %>%
  bind_rows(boris_synonyms %>%
              mutate(selected = "boris")) %>%
  group_by(selected) %>%
  top_n(15, similarity) %>%
  mutate(token = reorder(word, similarity)) %>%
  filter(token!=selected) %>%
  ggplot(aes(token, similarity, fill = selected)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~selected, scales = "free") +
  scale_fill_manual(values = c("#336B87", "#2A3132")) +
  coord_flip() +
  theme_tufte(base_family = "Helvetica")
```



```r
library(text2vec) # for implementation of GloVe algorithm
library(stringr) # to handle text strings
library(umap) # for dimensionality reduction later on
library(RcppParallel) # for parallel processing
```

```r
# ================================ choice parameters
# ================================
WINDOW_SIZE <- 6
DIM <- 300
ITERS <- 100
COUNT_MIN <- 10
```

```r
# shuffle text
set.seed(42L)
text <- sample(twts_sample$tweet)
```

```r
# ================================ create vocab ================================
tokens <- space_tokenizer(text)
it <- itoken(tokens, progressbar = FALSE)
vocab <- create_vocabulary(it)
vocab_pruned <- prune_vocabulary(vocab, term_count_min = COUNT_MIN)  # keep only words that me
et count threshold
```

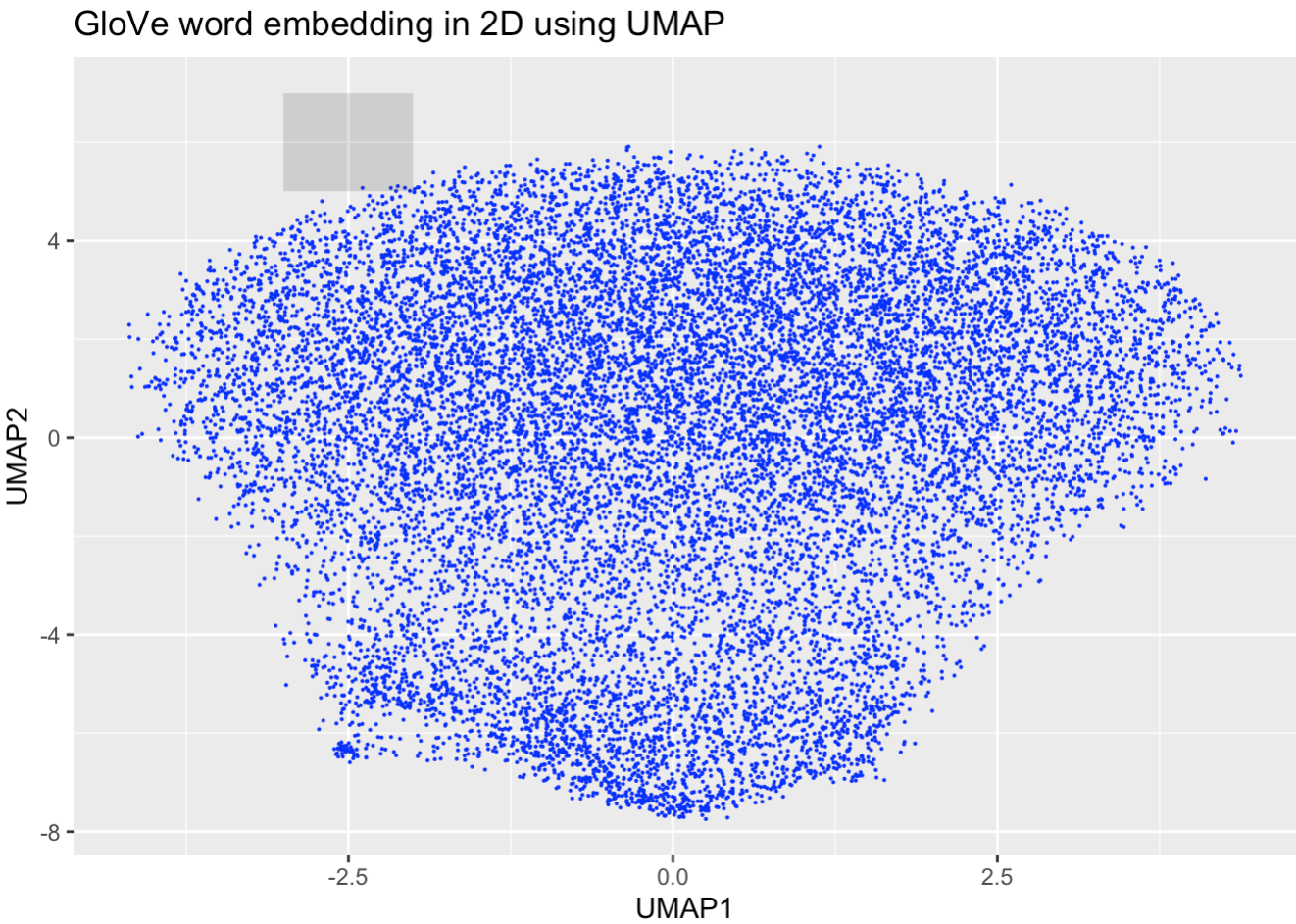
```r
# ================================ create term co-occurrence matrix
# ================================
vectorizer <- vocab_vectorizer(vocab_pruned)
tcm <- create_tcm(it, vectorizer, skip_grams_window = WINDOW_SIZE, skip_grams_window_context =
"symmetric",
    weights = rep(1, WINDOW_SIZE))
```

```r
# ================================ set model parameters
# ================================
glove <- GlobalVectors$new(rank = DIM, x_max = 100, learning_rate = 0.05)

# ================================ fit model ================================
word_vectors_main <- glove$fit_transform(tcm, n_iter = ITERS, convergence_tol = 0.001,
    n_threads = RcppParallel::defaultNumThreads())
```

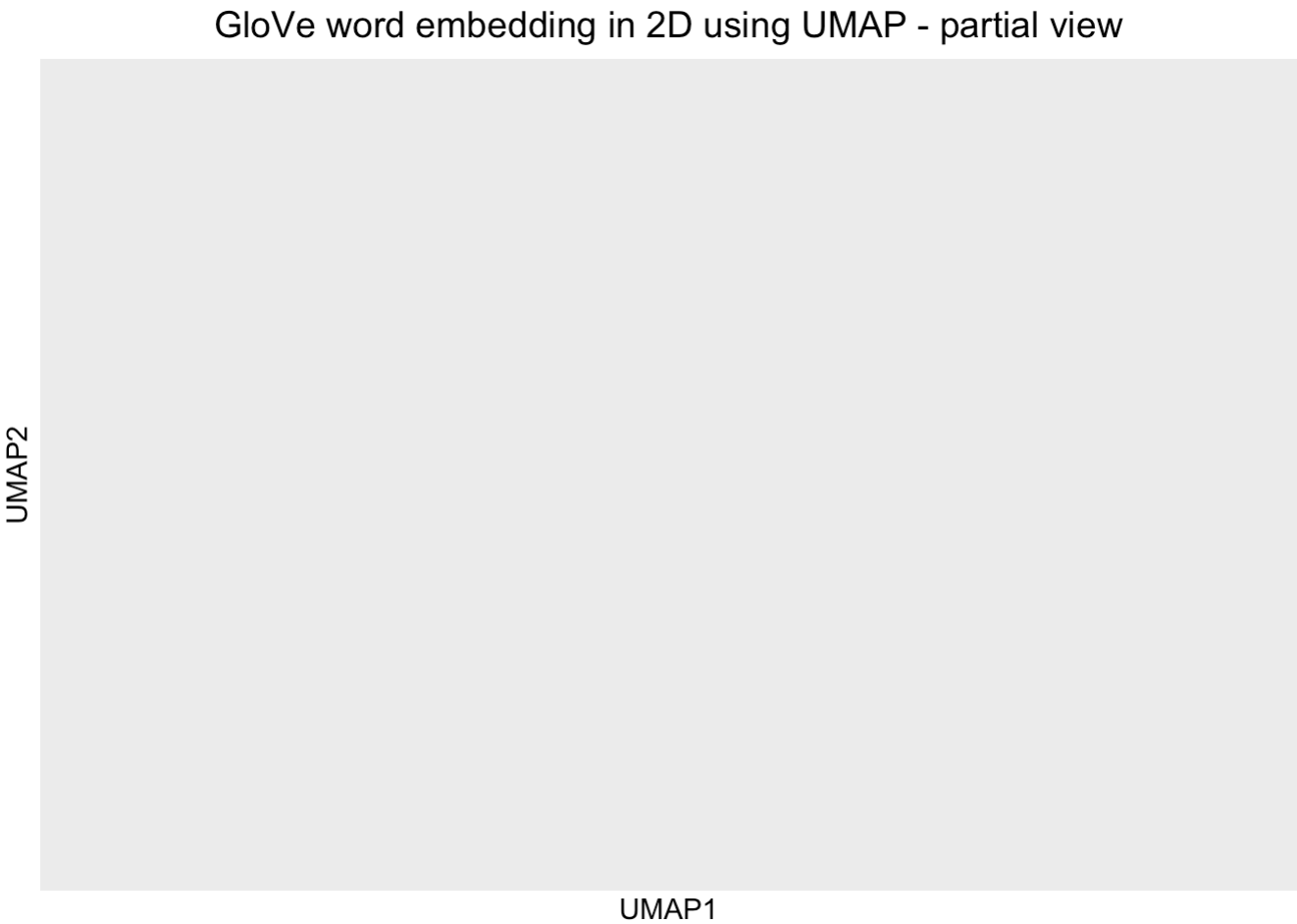
```r
# ================================ get output start from here================================
word_vectors_context <- glove$components
glove_embedding <- word_vectors_main + t(word_vectors_context)  # word vectors

# ================================ save ================================
saveRDS(glove_embedding, file = "local_glove.rds")
```

```r
# GloVe dimension reduction
glove_umap <- umap(glove_embedding, n_components = 2, metric = "cosine", n_neighbors = 25, min
_dist = 0.1, spread=2)
```

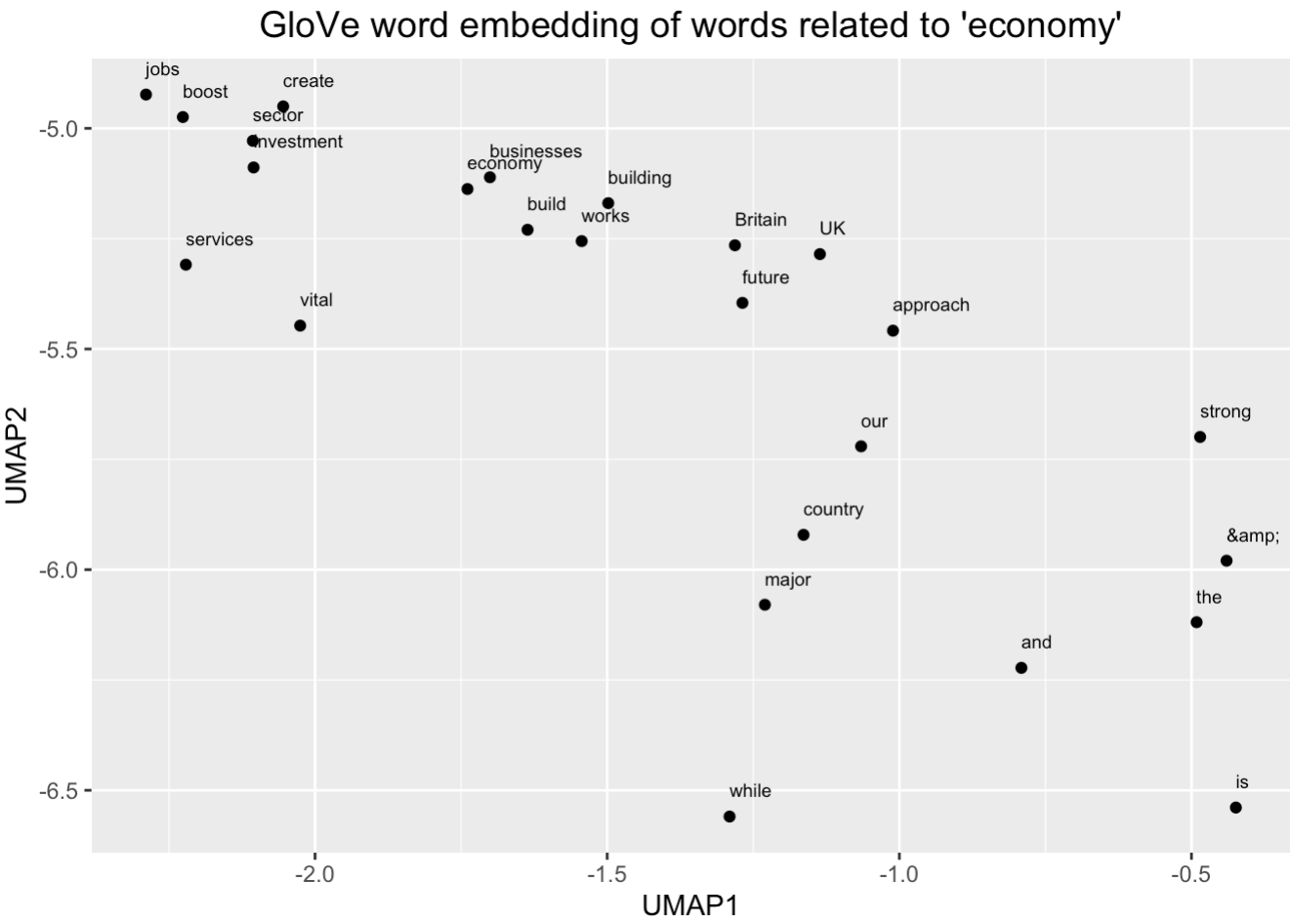
```
# Put results in a dataframe for ggplot
df_glove_umap <- as.data.frame(glove_umap[["layout"]])

# Add the labels of the words to the dataframe
df_glove_umap$word <- rownames(df_glove_umap)
colnames(df_glove_umap) <- c("UMAP1", "UMAP2", "word")

# Plot the UMAP dimensions
ggplot(df_glove_umap) +
  geom_point(aes(x = UMAP1, y = UMAP2), colour = 'blue', size = 0.05) +
  ggplot2::annotate("rect", xmin = -3, xmax = -2, ymin = 5, ymax = 7,alpha = .2) +
  labs(title = "GloVe word embedding in 2D using UMAP")
```




```
# Plot the shaded part of the GloVe word embedding with labels
ggplot(df_glove_umap[df_glove_umap$UMAP1 < -2.5 & df_glove_umap$UMAP1 > -3 & df_glove_umap$UMA
P2 > 5 & df_glove_umap$UMAP2 < 6.5,]) +
      geom_point(aes(x = UMAP1, y = UMAP2), colour = 'blue', size = 2) +
      geom_text(aes(UMAP1, UMAP2, label = word), size = 2.5, vjust=-1, hjust=0) +
      labs(title = "GloVe word embedding in 2D using UMAP - partial view") +
      theme(plot.title = element_text(hjust = .5, size = 14))
```



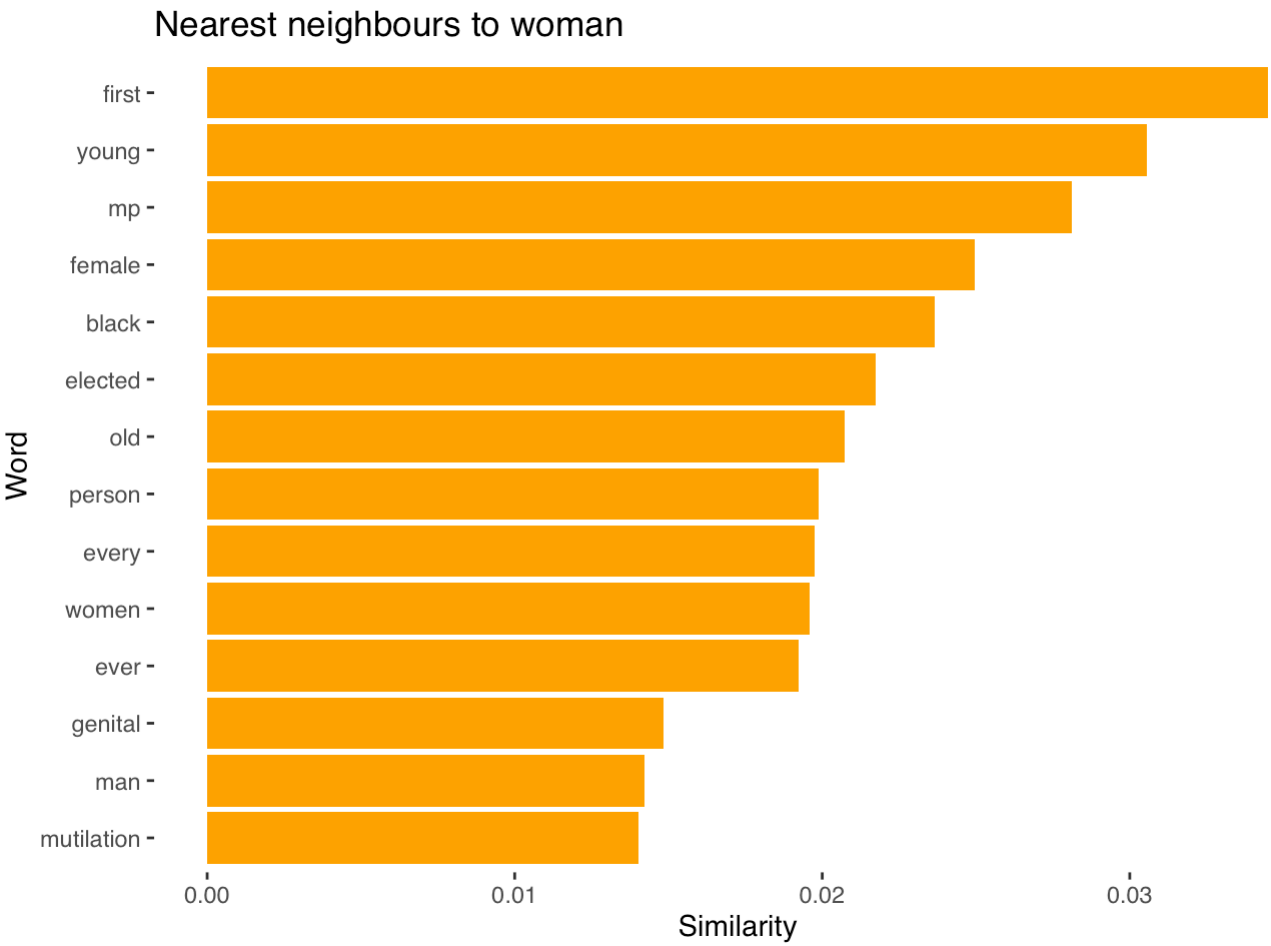

```
# Plot the word embedding of words that are related for the GloVe model
word <- glove_embedding["economy",, drop = FALSE]
cos_sim = sim2(x = glove_embedding, y = word, method = "cosine", norm = "l2")
select <- data.frame(rownames(as.data.frame(head(sort(cos_sim[,1], decreasing = TRUE), 25))))
colnames(select) <- "word"
selected_words <- df_glove_umap %>%
  inner_join(y=select, by= "word")

#The ggplot visual for GloVe
ggplot(selected_words, aes(x = UMAP1, y = UMAP2)) +
      geom_point(show.legend = FALSE) +
      geom_text(aes(UMAP1, UMAP2, label = word), show.legend = FALSE, size = 2.5, vjust=-1.5,
hjust=0) +
      labs(title = "GloVe word embedding of words related to 'economy'") +
      theme(plot.title = element_text(hjust = .5, size = 14))
```




```
#1
my_target_word <- "woman"

my_synonyms <- nearest_words(word_vectors, my_target_word)
```

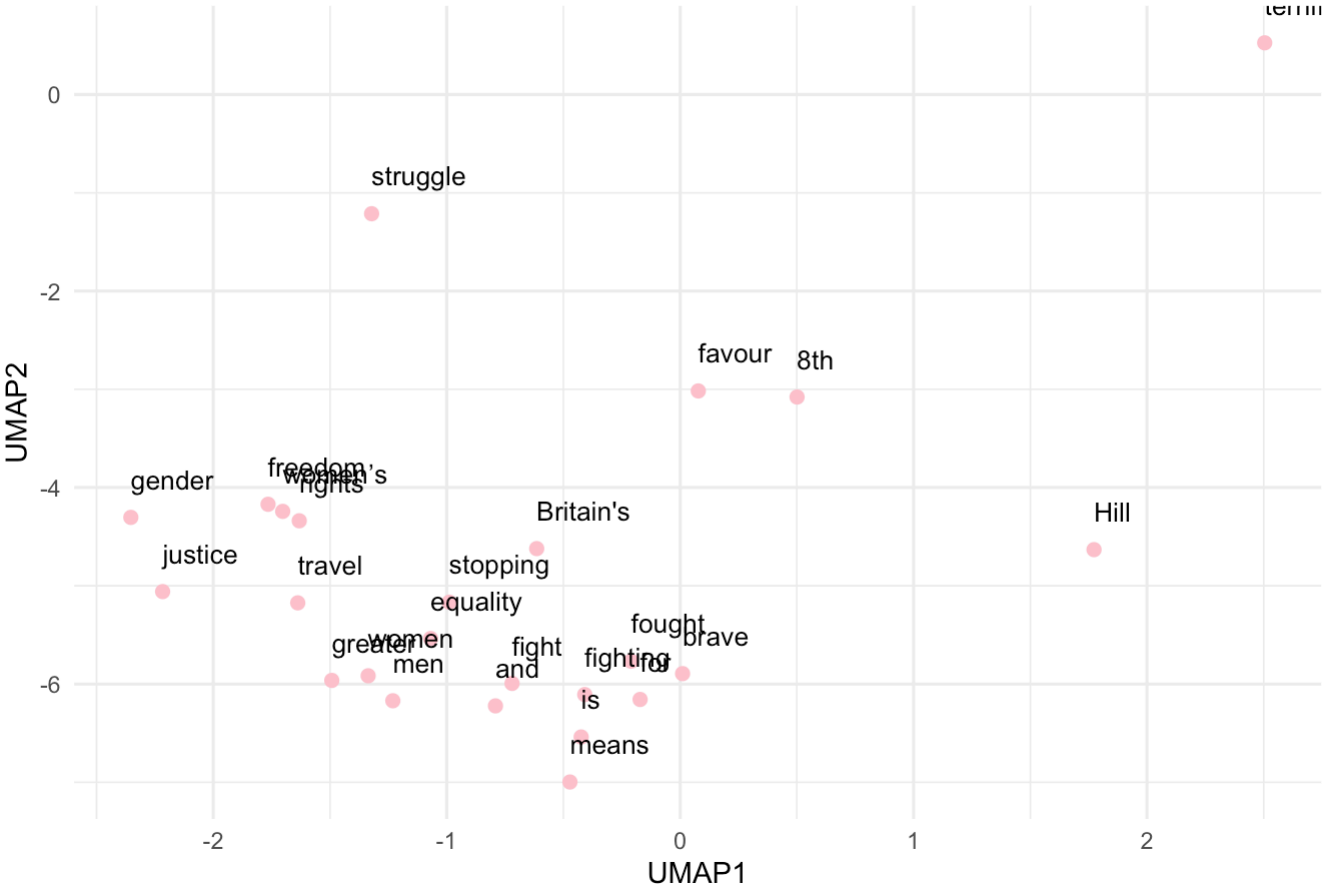
```
## Joining with `by = join_by(word)`
```

```
my_synonyms %>%
  mutate(selected = my_target_word) %>%
  top_n(15, similarity) %>% #extract the top 15 word
  mutate(token = reorder(word, similarity)) %>%
  filter(token != selected) %>%
  ggplot(aes(token, similarity, fill = selected)) +
  geom_col(show.legend = FALSE, fill = "orange") +
  coord_flip() +
  theme_tufte(base_family = "Helvetica") +
  labs(title = paste("Nearest neighbours to", my_target_word),
       x = "Word",
       y = "Similarity")
```



#2

```
new_interest <- "equality"

# searching for the most similar 25 words
word2 <- glove_embedding[new_interest, , drop = FALSE]
cos_sim2 = sim2(x = glove_embedding, y = word2, method = "cosine", norm = "l2")

select2 <- data.frame(rownames(as.data.frame(head(sort(cos_sim2[,1], decreasing = TRUE), 2
5))))
colnames(select2) <- "word"

#extract the 25 words from the 2D
selected_words2 <- df_glove_umap %>%
  inner_join(y = select2, by = "word")
```

```
#ggplot
ggplot(selected_words2, aes(x = UMAP1, y = UMAP2)) +
  geom_point(show.legend = FALSE, colour = "pink", size = 2) +
  geom_text(aes(UMAP1, UMAP2, label = word), show.legend = FALSE, size = 3.5, vjust=-1.5, hjus
t=0) +
  labs(title = paste("GloVe word embedding of words related to", new_interest)) +
  theme_minimal() +
  theme(plot.title = element_text(hjust = .5, size = 14))
```

GloVe word embedding of words related to equality